COMP 441 — Software Analysis and Testing

Week 9 — Performance, Usability and Exploratory Testing

Group A · Track D · TOML Kit

Period	Week 9
Evidence folder	docs/week9
Repository	github.com/the-dezeray/tomlkit
Commit(s)	see source report
Generated	2026-10-27
Contents	10 sections

This pack collects the week's human-readable report and key evidence. Raw AI-only logs are listed in the inventory and intentionally not printed.

Contents

Week 9 Light-Touch Performance and Usability Report . . . 3

1. Performance Check . . . 3
2. Usability Check . . . 3
3. Overall Conclusion . . . 5
Evidence Files . . . 5

Week 9 Exploratory Session Report . . . 5

Session Information . . . 5
Charter . . . 5
Initial Risks . . . 5
Session Log . . . 5
Session Conclusions . . . 7
AI Co-Pilot Evaluation . . . 7
Follow-Up Recommendations . . . 7
Session End . . . 7
Performance environment . . . 7
Performance results . . . 7
Usability check — error diagnosis . . . 8
Usability check — insertion at index 0 . . . 8
Usability check — modification . . . 8
Usability check — help/documentation . . . 9
Usability check — API misuse . . . 9
Performance harness . . . 9
Raw evidence inventory . . . 10

Source: docs\week9\performance_usability_report.md

Week 9 Light-Touch Performance and Usability Report

1. Performance Check

Objective

Measure a representative tomlkit workflow consisting of parsing, modifying, serializing, and reparsing a TOML document while verifying round-trip correctness.

Environment

- Timestamp: 2026-10-25 13:18:07 CAT
- Platform: Darwin 25.6.0 arm64
- Processor: Apple M2
- Logical CPUs: 8
- Python: 3.14.7
- tomlkit: 0.15.1

Method

- Operations per repeat: 5,000
- Repeats: 5
- Total operations: 25,000
- Each operation parsed a TOML document, changed a scalar value, appended an array value, serialized the document, reparsed the output, and asserted that the changed value survived the round trip.

Results

- Repeat 1: 2.130827 seconds, 2,346.51 operations/second
- Repeat 2: 2.122317 seconds, 2,355.92 operations/second
- Repeat 3: 2.075538 seconds, 2,409.01 operations/second
- Repeat 4: 2.076870 seconds, 2,407.47 operations/second
- Repeat 5: 2.072806 seconds, 2,412.19 operations/second
- Total measured time: 10.478358 seconds
- Mean repeat time: 2.095672 seconds
- Median repeat time: 2.076870 seconds
- Minimum repeat time: 2.072806 seconds
- Maximum repeat time: 2.130827 seconds
- Overall throughput: 2,385.87 operations/second
- Correctness check: all round-trip assertions passed

Interpretation and Limitations

The benchmark provides a light-touch performance baseline for one representative workload on the recorded machine. The five repeat results were similar, and every correctness assertion passed. The result is not a formal performance guarantee because it used one machine, one document shape, one process, and a short synthetic workload.

2. Usability Check

The usability observations and top three issues will be recorded after completing the developer-facing heuristic evaluation.

Evaluation Scope

A developer-facing heuristic evaluation was performed using common parsing, error-handling, array-modification, and help-discovery tasks. The evaluation considered Nielsen principles that are applicable to a Python library API, particularly consistency and standards, error prevention, visibility of outcomes, and help and documentation.

Positive Finding: Error Diagnosis

Four invalid TOML inputs were tested: a leading-zero integer, an unterminated string, an invalid escape, and a duplicate key. Each input produced a specific exception with a descriptive message and line and column information. This supports the heuristic of helping users recognize, diagnose, and recover from errors.

Top Three Usability Issues

Issue 1: Beginning insertion produces inconsistent array spacing

Heuristic: Consistency and standards.

For the conventionally formatted array `[1, 2]`, inserting `3` at position 0 produced `[3,1, 2]`. In comparison:

- Appending rendered `[1, 2, 3]`.
- Middle insertion rendered `[1, 3, 2]`.
- Replacing the first item rendered `[3, 2]`.
- Deleting the first item rendered `[2]`.

All outputs reparsed correctly, so the issue affects formatting consistency rather than semantic correctness.

Impact: Users relying on tomlkit to preserve human-readable formatting can receive inconsistent output after an otherwise normal array modification. Repeated beginning insertions can extend the inconsistent no-space section.

Severity: Medium.

Recommendation: Preserve the existing separator style when inserting before the first item. Add a regression test requiring insertion into `[1, 2]` to render as `[3, 1, 2]`.

Issue 2: Array insertion index clamping is not explained by built-in help

Heuristic: Help and documentation; match between the system and the real world.

The tested behavior was:

- Position `-99` inserted at the beginning.
- Position `-1` inserted before the final item.
- Position `99` inserted at the end.
- Every call returned `None`.

This is consistent with Python-style sequence insertion and is not classified as a functional defect. However, the built-in docstring only states `insert value before index` and does not explain how indexes outside the array bounds are handled.

Impact: A developer unfamiliar with Python list insertion rules may expect an out-of-range error or may be uncertain where the item was inserted.

Severity: Low.

Recommendation: Expand the `Array.insert()` documentation to state that indexes below the valid range are clamped to the beginning and indexes above the valid range are clamped to the end.

Issue 3: Common modification methods provide limited formatting guidance

Heuristic: Help and documentation.

The built-in help for `Array.insert()` and `Array.append()` consists of short sequence-style descriptions. The help does not explain whether these operations preserve existing whitespace, comments, commas, or multiline layout. In contrast, `Array.add_line()` has a longer docstring with parameters and an example showing exact rendered output.

Impact: Developers must experiment or inspect output to understand the formatting consequences of common modifications, even though formatting preservation is a central use case for tomlkit.

Severity: Low.

Recommendation: Add short formatting examples to the documentation for `insert()`, `append()`, replacement, and deletion. Clarify which existing formatting elements are expected to be preserved.

Usability Conclusion

The tested parser error messages were specific and actionable. Common array modifications also preserved semantic values and generated reparsable TOML. The principal usability weakness was inconsistent spacing after insertion at the beginning of a conventionally spaced array. Two lower-severity documentation issues concerned undisclosed index clamping and limited guidance about formatting preservation.

3. Overall Conclusion

The light-touch benchmark completed 25,000 parse-modify-serialize-reparse operations in 10.478358 seconds, with an overall throughput of 2,385.87 operations per second on the recorded Apple M2 environment. All round-trip correctness assertions passed.

The usability evaluation identified one medium-severity formatting-consistency issue and two low-severity documentation issues. Error diagnosis was a strength because the tested invalid inputs produced descriptive exceptions with line and column information.

Evidence Files

- `performance_environment.txt`
- `performance_check.py`
- `performance_results.txt`
- `usability_error_check.txt`
- `usability_api_misuse_check.txt`
- `usability_insert_check.txt`
- `usability_modification_check.txt`
- `usability_help_check.txt`

Source: docs\week9\exploratory_session_report.md

Week 9 Exploratory Session Report

Session Information

- Start: 2026-10-25 13:12:42 CAT
- Project: tomlkit
- Environment: Python 3.14.7
- Testing approach: Exploratory and ad-hoc testing with an AI co-pilot

Charter

Explore how tomlkit preserves array formatting during insertion, deletion, serialization, and reparsing, with particular attention to boundary indexes, unusual spacing, comments, and the known D8 spacing defect.

Initial Risks

- Array modifications may preserve values but damage user formatting.
- Boundary indexes may behave differently from middle indexes.
- Serialized output may remain valid TOML while losing expected spacing or comments.
- Repeated modifications may introduce cumulative formatting changes.

Session Log

Test 1: Compare insertion positions

AI co-pilot suggestion: Compare insertion at the beginning, middle, and end of the same array, then reparse each serialized result.

Result:

- Inserting at position 0 rendered `[0,1, 2]`, with the expected space missing after the first comma.
- Inserting at position 1 rendered `[1, 0, 2]`.
- Inserting at position 2 rendered `[1, 2, 0]`.

- All three serialized arrays reparsed to the correct values.

Observation: D8 was reproduced. The observed defect affects formatting when inserting at the beginning boundary, while the tested middle and end positions retained consistent spacing. Semantic array values and reparsing remained correct in all three cases.

AI suggestion assessment: Useful. Comparing boundary and non-boundary positions narrowed the observed behavior to insertion at position 0 for this input.

Test 2: Vary the original spacing style

AI co-pilot suggestion: Repeat insertion at position 0 using compact, conventionally spaced, bracket-spaced, and unusually spaced source arrays.

Results:

- `[1, 2]` rendered as `[0,1, 2]`.
- `[1,2]` rendered as `[0,1,2]`.
- `[ 1, 2 ]` rendered as `[ 0, 1, 2 ]`.
- `[ 1 , 2 ]` rendered as `[ 0, 1 , 2 ]`.
- Every result reparsed to `[0, 1, 2]`.

Observation: The formatting issue was not reproduced for every insertion at position 0. It appeared for the conventionally spaced source `[1, 2]`, where the inserted value was followed by no space while the existing separator retained a space. The compact source remained consistently compact, and sources beginning with whitespace after the opening bracket retained a spaced style. All tested outputs remained valid TOML and semantically correct.

Follow-up question: Is the first element's leading whitespace reused to format the inserted element rather than being transferred or copied to the original first element?

Test 3: Insert into a multiline array containing comments

Test: Insert 0 at position 0 in a multiline array whose two existing elements have inline comments, then serialize and reparse the document.

Results:

- The inserted value was placed on a new, consistently indented line.
- The comments `# first` and `# second` were both preserved.
- The modified array had values `[0, 1, 2]`.
- The serialized document reparsed to `[0, 1, 2]`.

Observation: The single-line D8 spacing defect was not reproduced in this multiline case. Beginning insertion preserved the multiline layout, comments, semantic values, and reparsability for the tested input.

Test 4: Repeat insertion at the beginning boundary

AI co-pilot suggestion: Insert several values consecutively at position 0 to determine whether the formatting issue remains isolated or accumulates.

Results:

- After inserting 0, the array rendered as `[0,1, 2]`.
- After inserting -1, the array rendered as `[-1,0,1, 2]`.
- After inserting -2, the array rendered as `[-2,-1,0,1, 2]`.
- Every serialized result reparsed to the expected values.

Observation: Repeated insertion at position 0 caused cumulative formatting degradation. Each new value extended the compact, no-space portion at the beginning of the array, while the original space before `2` remained. Semantic values and reparsability were preserved throughout.

Risk: Repeated edits can make a conventionally spaced array increasingly inconsistent even though the resulting TOML remains valid.

Test 5: Delete inserted values and check formatting recovery

AI co-pilot suggestion: After reproducing cumulative formatting degradation, delete the inserted values to test whether the original formatting remains damaged.

Results:

- Original array: `[1, 2]`.
- After three beginning insertions: `[-2,-1,0,1, 2]`.
- Successive deletions rendered `[-1,0,1, 2]`, then `[0,1, 2]`, and finally `[1, 2]`.
- Every intermediate result reparsed to the expected values.
- The final rendering matched the original array exactly.

Observation: The formatting inconsistency accumulated while inserted values remained in the array, but deleting those values restored the original formatting exactly for this input.

Rejected concern: The initial concern that beginning insertion might permanently damage the original array formatting was not supported by this test. The concern was useful as a test idea but would have been misleading if reported as a conclusion without verification.

Session Conclusions

1. The known D8 formatting defect was reproduced for insertion at position 0 in the conventionally spaced single-line array `[1, 2]`.
2. The defect affected formatting, while semantic values and TOML reparsability remained correct in every executed test.
3. Middle and end insertion did not reproduce the observed missing-space behavior.
4. Beginning insertion preserved consistent formatting for the compact array `[1,2]` and for tested arrays containing whitespace after the opening bracket.
5. The defect was not reproduced in the tested multiline array, and both inline comments were preserved.
6. Repeated beginning insertions caused cumulative formatting inconsistency.
7. Deleting all inserted values restored the original `[1, 2]` formatting exactly.

AI Co-Pilot Evaluation

Useful assistance: The suggestion to compare beginning, middle, and end insertion positions narrowed the behavior to the position-0 boundary for the initial input. The suggestion to repeat beginning insertion exposed cumulative formatting degradation.

Potentially misleading idea: The possibility that insertion permanently damages existing formatting was treated only as a hypothesis. Test 5 disproved that concern for the tested input, demonstrating why AI-generated ideas must be verified rather than reported as facts.

Follow-Up Recommendations

- Add a regression test requiring insertion at position 0 into `[1, 2]` to render as `[0, 1, 2]`.
- Retain semantic round-trip assertions alongside exact-format assertions.
- Test additional element types, nested arrays, and arrays with comments before generalizing the findings.
- Investigate the handling of whitespace trivia around the original first element.
- Keep D8 documented as a formatting-preservation defect rather than a semantic parsing failure.

Session End

- End: 2026-10-25 13:16:51 CAT

Performance environment

Source: docs\week9\performance_environment.txt

```
Timestamp: 2026-10-25 13:18:07 CAT
Platform: Darwin 25.6.0 arm64
Processor: Apple M2
Logical CPUs: 8
Python: Python 3.14.7
tomlkit: 0.15.1
```

Performance results

Source: docs\week9\performance_results.txt

```
Light-touch tomlkit performance check
Operations per repeat: 5000
```

```
Repeats: 5
Workload: parse, modify, serialize, and reparse
Repeat 1: 2.130827 seconds, 2346.51 operations/second
Repeat 2: 2.122317 seconds, 2355.92 operations/second
Repeat 3: 2.075538 seconds, 2409.01 operations/second
Repeat 4: 2.076870 seconds, 2407.47 operations/second
Repeat 5: 2.072806 seconds, 2412.19 operations/second
Total operations: 25000
Total measured time: 10.478358 seconds
Mean repeat time: 2.095672 seconds
Median repeat time: 2.076870 seconds
Minimum repeat time: 2.072806 seconds
Maximum repeat time: 2.130827 seconds
Overall throughput: 2385.87 operations/second
Correctness check: all round-trip assertions passed
```

Usability check — error diagnosis

Source: docs\week9\usability_error_check.txt

```
USABILITY CHECK 1: Error feedback for invalid TOML

Case: leading-zero integer
Input: 'value = 01'
Exception type: InvalidNumberError
Message: Invalid number at line 1 col 10
Position attributes: line=1, col=10

Case: unterminated string
Input: 'value = "abc'
Exception type: UnexpectedEofError
Message: Unexpected end of file at line 1 col 12
Position attributes: line=1, col=12

Case: invalid escape
Input: 'value = "a\\q"'
Exception type: InvalidCharInStringError
Message: Invalid character 'q' in string at line 1 col 11
Position attributes: line=1, col=11

Case: duplicate key
Input: 'value = 1\nvalue = 2'
Exception type: ParseError
Message: Key "value" already exists. at line 2 col 9
Position attributes: line=2, col=9
```

Usability check — insertion at index 0

Source: docs\week9\usability_insert_check.txt

```
USABILITY CHECK 3: Array insertion index behavior

position=-99
return_value=None
values=[3, 1, 2]
rendered='[3,1, 2]'

position=-1
return_value=None
values=[1, 3, 2]
rendered='[1, 3, 2]'

position=0
return_value=None
values=[3, 1, 2]
rendered='[3,1, 2]'

position=2
return_value=None
values=[1, 2, 3]
rendered='[1, 2, 3]'

position=99
return_value=None
values=[1, 2, 3]
rendered='[1, 2, 3]'
```

Usability check — modification

Source: docs\week9\usability_modification_check.txt

```
USABILITY CHECK 4: Formatting after common array modifications

Operation: append
rendered='[1, 2, 3]'
values=[1, 2, 3]
reparsed=[1, 2, 3]

Operation: insert at beginning
rendered='[3,1, 2]'
values=[3, 1, 2]
reparsed=[3, 1, 2]

Operation: insert in middle
rendered='[1, 3, 2]'
values=[1, 3, 2]
reparsed=[1, 3, 2]

Operation: replace first item
rendered='[3, 2]'
values=[3, 2]
reparsed=[3, 2]

Operation: delete first item
rendered='[2]'
values=[2]
reparsed=[2]
```

Usability check — help/documentation

Source: docs\week9\usability_help_check.txt

```
USABILITY CHECK 5: Discoverability and built-in help

Method: Array.insert
Signature: (self, pos: 'int', value: 'Any') -> 'None'
Docstring: 'S.insert(index, value) -- insert value before index'

Method: Array.append
Signature: (self, value)
Docstring: 'S.append(value) -- append value to the end of the sequence'

Method: Array.add_line
Signature: (self, *items: 'Any', indent: 'str' = '    ', comment: 'str | None' = None, add_comma: 'bool' = True, newline
: 'bool' = True) -> 'None'
Docstring: 'Add multiple items in a line to control the format precisely.\nWhen add_comma is True, only accept actual va
lues and\n", " will be added between values automatically.\n\n:Example:\n\n>>> a = array()\n>>> a.add_line(1, 2, 3)\n>>>
 a.add_line(4, 5, 6)\n>>> a.add_line(indent="")\n>>> print(a.as_string())\n[\n    1, 2, 3,\n    4, 5, 6,\n]'
```

Usability check — API misuse

Source: docs\week9\usability_api_misuse_check.txt

```
USABILITY CHECK 2: Feedback for common API misuse

Case: access missing document key
Exception type: NonExistentKey
Message: 'Key "missing" does not exist.'

Case: insert beyond array length
Result: accepted, returned None

Case: add unsupported object type
Exception type: ConvertError
Message: Unable to convert an object of <class 'object'> to a TOML item

Case: add multiline array comment
Exception type: ValueError
Message: Comment cannot contain line breaks
```

Performance harness

Source: docs\week9\performance_check.py

```
import statistics
import time

import tomlkit


SOURCE = """
title = "COMP 441 benchmark"
enabled = true
```

```
count = 42
tags = ["testing", "mutation", "bdd"]

[database]
host = "localhost"
port = 5432

[[students]]
name = "Student 1"
score = 75

[[students]]
name = "Student 2"
score = 82
""".lstrip()

OPERATIONS = 5_000
REPEATS = 5


def workload():
    for i in range(OPERATIONS):
        document = tomlkit.parse(SOURCE)
        document["count"] = i
        document["tags"].append("performance")
        rendered = document.as_string()
        reparsed = tomlkit.parse(rendered)

        if reparsed["count"] != i:
            raise AssertionError("Round-trip value mismatch")


durations = []

print("Light-touch tomlkit performance check")
print(f"Operations per repeat: {OPERATIONS}")
print(f"Repeats: {REPEATS}")
print("Workload: parse, modify, serialize, and reparse")

for repeat in range(1, REPEATS + 1):
    start = time.perf_counter()
    workload()
    duration = time.perf_counter() - start
    durations.append(duration)

    rate = OPERATIONS / duration
    print(
        f"Repeat {repeat}: "
        f"{duration:.6f} seconds, "
        f"{rate:.2f} operations/second"
    )

total_operations = OPERATIONS * REPEATS
total_duration = sum(durations)

print(f"Total operations: {total_operations}")
print(f"Total measured time: {total_duration:.6f} seconds")
print(f"Mean repeat time: {statistics.mean(durations):.6f} seconds")
print(f"Median repeat time: {statistics.median(durations):.6f} seconds")
print(f"Minimum repeat time: {min(durations):.6f} seconds")
print(f"Maximum repeat time: {max(durations):.6f} seconds")
print(
    "Overall throughput: "
    f"{total_operations / total_duration:.2f} operations/second"
)
print("Correctness check: all round-trip assertions passed")
```

Raw evidence inventory

The pack prints the readable report and key summaries. Large raw artefacts are kept in the repository and listed here so a marker can verify them without paging through thousands of lines. Items tagged **[AI-only]** are deliberately not printed: they are the AI baseline's raw output, not the final hybrid result.

File	Size	Tag
docs\week9\exploratory_session_report.md	7 KB	
docs\week9\performance_check.py	2 KB	
docs\week9\performance_environment.txt	145 B	
docs\week9\performance_results.txt	721 B	

File	Size	Tag
docs\week9\performance_usability_report.md	7 KB	
docs\week9\usability_api_misuse_check.txt	481 B	
docs\week9\usability_error_check.txt	752 B	
docs\week9\usability_help_check.txt	846 B	
docs\week9\usability_insert_check.txt	418 B	
docs\week9\usability_modification_check.txt	487 B	